

智慧医疗对话系统软件设计报告

学号：22320021 姓名：陈安康

一、文档概述

本文档详细描述了基于 Dify 平台构建的智慧医疗对话系统的软件架构设计与实现方案。系统以自然语言处理与知识问答为核心，融合大语言模型（LLM）、知识库检索、用户界面与后端服务等关键模块，实现针对糖尿病等慢性病的智能问答与健康辅导功能。文档涵盖系统功能、模块设计、用例建模、接口交互等。

本设计文档旨在为项目开发、测试、维护人员提供清晰的技术规范与参考依据，并支持系统的持续迭代优化。

二、系统概述

智慧医疗对话系统基于开源 LLM 平台 Dify 实现，采用前后端分离架构，融合医疗知识库与用户健康交互场景。主要功能包括：

用户通过 Web 或移动端界面输入健康问题；

系统调用知识库服务进行语义匹配与文档搜索；

后端自动构造 Prompt，将用户问题和文档上下文传入大语言模型；

模型生成基于知识的专业问答，并返回给用户界面；

后台记录用户问题及问答结果，以用于个性化推荐和持续优化。

系统主要包含以下关键组件：

前端交互层（Web UI）：提供问答输入界面、答案显示、会话管理等功能；

Dify 后端引擎：处理对话逻辑、Prompt 构造、模型调用与知识注入；

知识服务模块：基于向量检索（如 FAISS）实现知识文档匹配；

大语言模型（如 Qwen-2.5-0.5B）：用于回答用户健康问题；

数据库（MySQL）：用于存储用户问题、答案记录与知识文档索引。

该系统支持私有化部署，可扩展其他疾病知识库，具备良好的通用性与可维护性。

三、设计目标和原则

本系统的设计围绕以下目标与原则展开：

1. 智能化与专业性

通过集成医疗知识库与大语言模型，实现对糖尿病等慢性病问题的专业解答，提升系统智能水平与医学准确性。

2. 模块化与可扩展性

系统采用模块化设计，知识库、LLM 模型、对话逻辑均为独立服务，便于后期扩展为多病种支持、多语言接口或与其他诊疗系统集成。

3. 用户友好性

界面设计简洁直观，支持语义补全、上下文追踪、历史问答查看等功能，降低用户使用门槛，提升交互体验。

4. 数据安全性与隐私保护

所有用户健康数据和问答记录均加密存储，符合国家健康数据安全规范，可选本地部署确保隐私安全。

5. 易部署与可维护性

系统支持 Docker 化部署，前后端解耦，配置灵活，便于运维管理与模型更新。

四、设计约束和现实限制

在系统开发与部署过程中，为确保功能实现与系统落地，需要考虑若干实际约束与限制因素。此部分列举并分析当前智慧医疗对话系统在设计与实现阶段所面临的关键约束与现实挑战。

4.1 设计约束

(1) 依赖 Dify 平台框架

系统整体基于 Dify 平台进行开发，因此需遵循其插件、模型调用、Prompt 模板等框架规范，限制了部分深度定制能力。

(2) LLM 模型兼容性

当前系统默认使用 Qwen-2.5-0.5B 等国产开源语言模型，模型调用接口需遵循 OpenAI 格式。如需替换为其他模型（如 ChatGLM、Baichuan），需调整接口适配器，增加部署复杂度。

(3) 向量检索方式固定

知识注入机制依赖向量化检索（如 FAISS），目前支持的向量模型和索引方式有限，需在准确率与召回率之间权衡，影响问答质量。

(4) 私有化部署需求

系统需支持本地部署，不能使用云端 LLM API（如 OpenAI、Claude），因此计算资源限制较为明显，影响模型规模选择与响应速度。

(5) 统一语言与术语标准

出于医疗专业性的要求，系统中疾病名称、用药名、症状描述等需参照权威标准（如 ICD-10、SNOMED CT），限制了问答内容生成的自由度。

4.2 现实限制

(1) 模型能力有限

Qwen-2.5-0.5B 等中小模型在医疗问答场景下尚存在推理不足、事实偏差、回答片面等问题，需依赖知识库提供强引导，降低 hallucination 现象。

(2) 知识库覆盖度不均

当前知识库仅覆盖糖尿病基础知识、并发症、防治方法，尚未包括罕见情况、最新研究成果，可能造成问答结果不完整或过时。

(3) 用户表达方式多样

用户输入的自然语言问题可能存在口语化、错别字、语义歧义等问题，需依赖较强的语义理解能力，而当前系统对此仍有局限。

(4) 数据保护政策限制

智慧医疗系统涉及用户健康信息，需遵循《个人信息保护法》《网络安全法》等法规，数据采集、存储、传输均需合规，限制了部分日志记录与推荐机制。

(5) 终端资源限制

在移动端部署场景下，设备内存与算力受限，可能无法直接运行完整模型，需通过边缘计算或推理加速方案进行优化。

五、体系结构设计

为保证智慧医疗对话系统的模块化设计、良好的可维护性及高可扩展性，系统采用经典的三层架构模式，分别为用户界面层（UI Layer）、业务逻辑层（Business Logic Layer）和基础服务层（Infrastructure Layer）。各层职责明确，层与层之间的耦合度低，便于功能的独立升级和模型的灵活替换。

1. 用户界面层（User Interface Layer）

用户界面层主要负责与最终用户交互。系统通过 `UIClient` 模块提供友好的交互界面，支持用户输入医疗咨询问题，并展示系统生成的智能回答。该层聚焦于用户体验设计，确保交互的流畅与实时响应。

2. 业务逻辑层（Business Logic Layer）

业务逻辑层是系统的核心，承担对用户请求的处理和智能推理功能，主要由以下模块组成：

APIGateway: 作为系统的入口，负责接收来自 UI 层的请求，协调后端各模块完成任务，并将结果返回给前端。

PromptManager: 负责构建和管理发送给大语言模型的 Prompt 文本，确保 Prompt 内容充分融合用户输入和相关知识信息。

KnowledgeInjector: 负责从知识库中检索与用户问题相关的知识向量，辅助模型生成更准确、专业的回答。

LLMEngine: 负责调用大语言模型进行推理计算，基于输入的 Prompt 生成智能回复。

SessionManager: 管理用户会话状态和历史对话记录，保证对话的上下文连续性，提升系统响应的针对性和个性化。

3. 基础服务层（Infrastructure Layer）

基础服务层提供系统运行所需的底层资源和数据支持，包括：

LLMBackend: 承载和管理大语言模型的计算资源，提供高效的模型推理服务接口。

VectorDB: 存储和管理知识库的向量化数据，支持快速相似度检索，保证知识注入模块的查询效率。

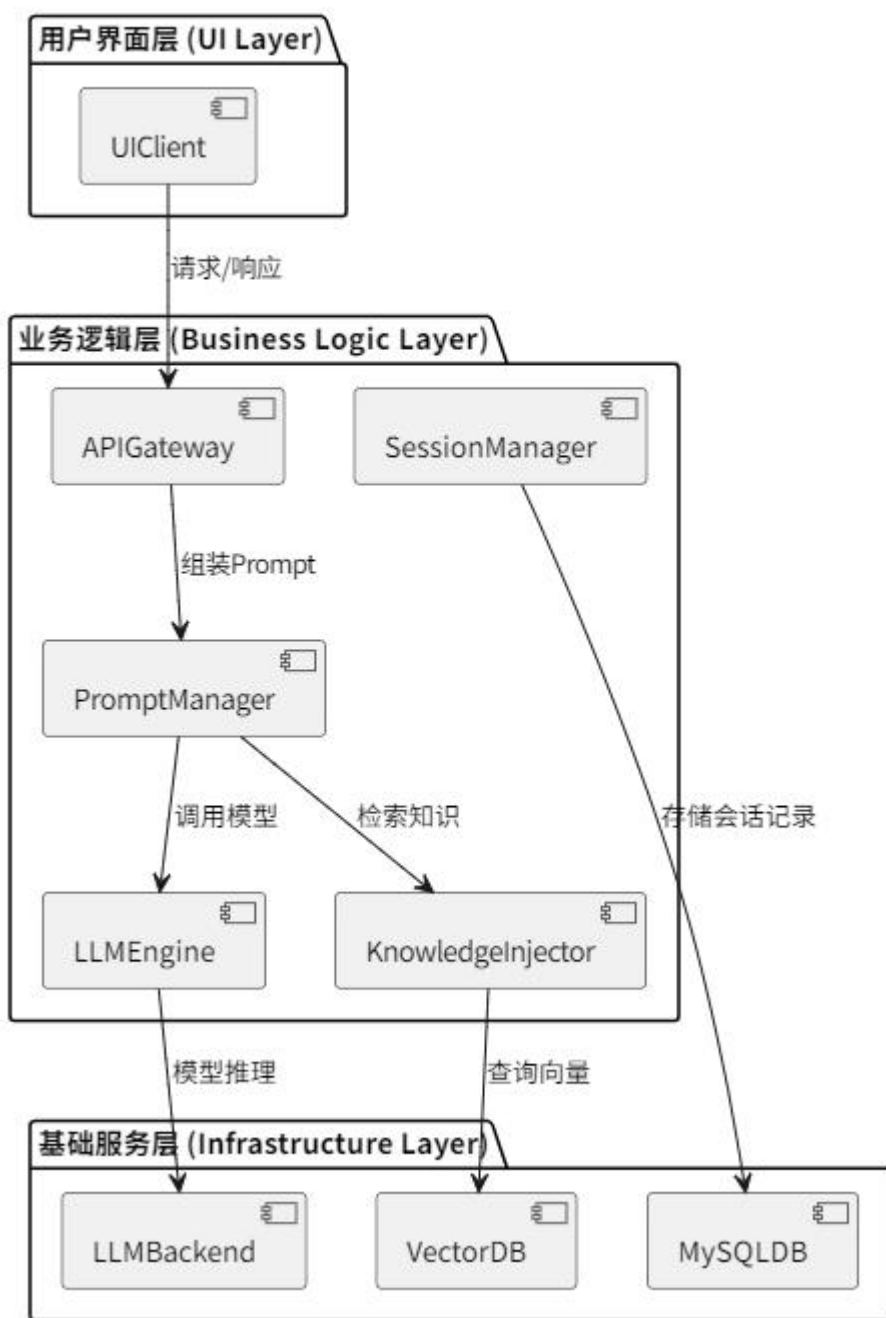
MySQLDB: 关系型数据库，用于存储系统的持久化数据，如用户会话记录、系统配置等。

各层交互流程

用户通过 `UIClient` 提交咨询请求，`APIGateway` 接收后由 `PromptManager` 负责整合用户输入和检索到的知识（通过 `KnowledgeInjector` 查询 `VectorDB`），构建完整的 **Prompt** 发送至 `LLMEngine`。`LLMEngine` 通过 `LLMBackend` 执行推理，生成回答返回至 `APIGateway`，最终反馈给用户界面。同时，`SessionManager` 将对话过程中的重要信息和会话状态存入 `MySQLDB`，为后续会话和数据分析提供支持。

整体架构设计既保障了系统响应的高效性，也为未来模型升级、多疾病知识库扩展及联网搜索功能的集成奠定了坚实基础。

智慧医疗对话系统软件体系结构



六、用户界面设计

为提升智慧医疗对话系统的交互效率与用户体验，前端界面遵循“简洁、直观、专业”的设计理念，结合典型医疗咨询场景与用户习惯，构建了多终端支持的图形用户界面（GUI）。该界面通过浏览器或移动端访问，为用户提供健康问答入口、对话记录管理和可视化反馈功能。

6.1 功能组成

前端界面主要包括以下几个核心模块：

问答交互区（QA Console）

输入框：支持多行自然语言输入，具备语义提示功能；

发送按钮：触发后将问题提交至后端处理；

回答区域：以气泡样式展示模型回复内容，支持高亮显示关键词、医学术语解释悬浮提示等；

回复来源提示：标注答案是否引用知识库内容或来自模型生成（可选择展开原文出处）。

历史会话区（Session Panel）

展示当前用户与系统的所有历史问答记录；

支持按时间、关键词筛选历史对话；

用户可点击任一历史对话节点，重新载入上下文继续提问。

用户信息与设置区（User Settings）

显示用户基本资料（昵称、性别、年龄等）；

提供个性化设定选项，如是否开启语音输入、字体大小、界面配色方案等；

支持会话清除、导出历史对话记录等功能。

健康辅助模块（Health Assistant Panel）

内嵌健康建议展示区：在问答之外，主动推荐与糖尿病管理相关的知识、饮食建议、锻炼计划等；

快捷入口：提供“常见问题”“并发症预防”“血糖记录”等快速问答按钮，降低用户操作门槛；

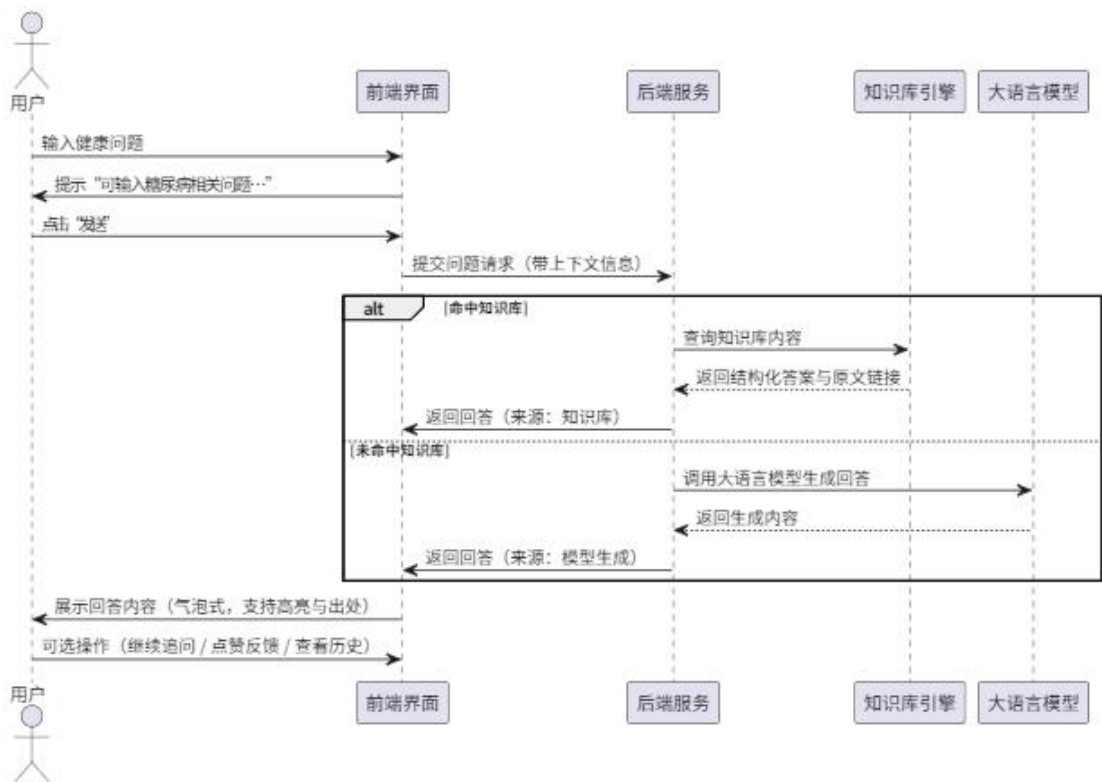
可选日历集成功能，用于记录个人健康日志与系统建议。

系统状态与反馈模块

显示系统当前运行状态（如“正在生成回答...”、“知识库查询中...”等）；

提供用户反馈入口，支持报告错误问答、提交建议等，便于系统持续优化。

6.2 交互流程示意



6.3 响应式设计 with 终端适配

PC 端：采用网格化布局，左侧为历史会话栏，右侧为问答主视图，适合长时间阅读与交互；

移动端：界面自动适配小屏幕尺寸，采用卡片式滚动显示结构，底部悬浮输入框优化单手操作体验；

可访问性：支持键盘操作与屏幕阅读器，符合 WCAG（Web Content Accessibility Guidelines）标准。

6.4 技术实现要点

基于现代前端框架（如 Vue.js 或 React）实现组件化开发；

使用 WebSocket 或轮询方式实现长对话时的流式响应；

与后端 API（通过 RESTful 或 GraphQL）无缝集成，异步加载问答内容；

结合 CSS3 动效和图标库（如 iconfont、fontawesome）提升视觉体验。

6.5 用户界面设计原则

以用户为中心：界面操作应贴合医疗咨询场景，语言用词应通俗易懂且保持专业；

信息安全可控：用户隐私信息不在前端明文展示，所有敏感数据仅作本地展示与加密传输；

上下文保留机制：每次提问自动记录上下文会话 ID，并保留历史输入内容；

反馈闭环机制：用户反馈路径清晰、简洁，提交后可获得处理状态反馈。

7 子系统/构件设计

为保证系统的可扩展性、模块化与易维护性，本系统采用分层架构设计，将整体系统划分为多个功能子系统与构件单元，明确职责边界与接口规范，支持独立部署与演化升级。整体架构如下图所示：

7.1 系统构架总览

系统整体分为五个核心子系统：

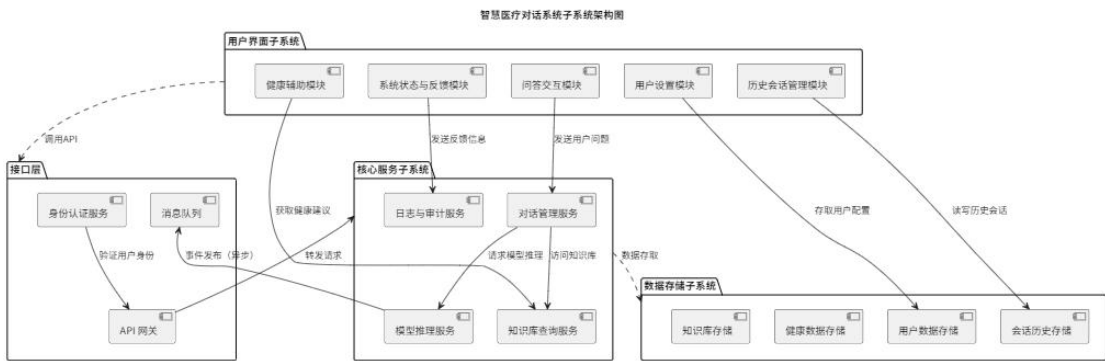
用户交互子系统（前端）

对话管理子系统（Dify workflow核心）

知识库子系统（本体、资料、文档解析与搜索）

大模型服务子系统（LLM 生成与增强）

系统管理与监控子系统



7.2 子系统与构件详细说明

7.2.1 用户交互子系统 (UI)

主要构件：

问答交互模块 (QA Console)

历史记录模块 (Session Panel)

设置与用户信息模块

健康推荐助手 (Health Assistant)

状态与反馈组件 (Status/Feedback)

接口说明：

与后端通过 RESTful API / WebSocket 通信；

接收 JSON 格式回答，支持富文本与元数据展示（高亮、解释、来源）；

支持响应式布局与本地缓存机制。

7.2.2 对话管理子系统

核心构件：

会话调度器：根据用户输入决定调用知识库或大模型；

上下文维护器：跟踪每一轮对话上下文，构造提示词；

反馈处理器：接收前端反馈信息并持久化；

用户状态管理器：绑定用户个性设置（年龄、语音输入等）与对话上下文。

技术特点：

使用 Dify Workflow 引擎编排 Prompt Flow；

内建 Prompt 模板机制与回退策略。

7.2.3 知识库子系统

模块组成：

知识文档解析器：支持 Markdown、PDF、网页等医疗资料导入；

向量化服务：使用 OpenAI Embedding / FastText 生成向量；

索引服务：采用 FAISS / Milvus 实现向量检索；

结果融合器：对检索结果进行打分、压缩、去冗余后返回文本。

接口协议：

提供标准 REST API：支持 query 参数与语义扩展；

内部支持基于问句的 Top-k 模糊匹配与关键词精确匹配组合。

7.2.4 大语言模型子系统

构件组成：

Prompt 编排模块：构造带上下文、知识融合的提示词；

模型服务接入器：对接 GPT-4、Claude、国内大模型 API；

回答优化器：执行结构化输出、关键词抽取、医疗术语解释生成；

安全控制模块：过滤敏感词、确保合规回复（如不诊断、不建议用药）。

补充能力：

支持多轮会话记忆；

可配置温度、top_p 等采样策略；

支持模型选择动态切换。

7.2.5 系统管理与监控子系统

构件模块：

日志记录器：记录所有用户对话与模型调用日志；

错误检测器：自动标记响应失败、知识库未命中等异常情况；

性能监控仪表盘：追踪模型响应延时、召回准确率等指标；

反馈处理队列：异步处理用户举报与建议。

运维支持：

可视化监控平台（如 Prometheus + Grafana）；

管理后台提供知识库增删改、模型 API 密钥配置等功能。

7.3 构件间通信与数据流

系统中关键数据流如下：

用户提交自然语言问题，前端打包上下文与用户 ID 发给后端；

对话管理子系统判断是否召回知识库（召回成功则融合），否则调用模型；

模型或知识库返回内容经由后端封装后，异步推送至前端；

用户操作与反馈同步写入日志与反馈处理队列。

7.4 模块间依赖与可扩展性设计

所有模块使用接口与协议通信，便于替换与扩展（如知识库可切换为其他搜索引擎）；

UI 模块可独立适配其他终端，如小程序、App 内嵌 Webview；

模型服务可按策略调用不同模型，支持 A/B 测试；

Dify 工作流为核心，支撑整个系统逻辑处理的灵活性与复用性。

8. 用例设计

本部分围绕智慧医疗对话系统的核心功能，设计主要用例以指导系统开发与测试。用例涵盖用户与系统的典型交互场景，确保系统功能的完整性与实用性。

8.1 主要参与者

用户：普通终端用户，通过前端界面咨询健康医疗问题，管理对话历史，调整个人设置。

系统管理员：负责维护系统后台，管理知识库和日志，处理用户反馈。

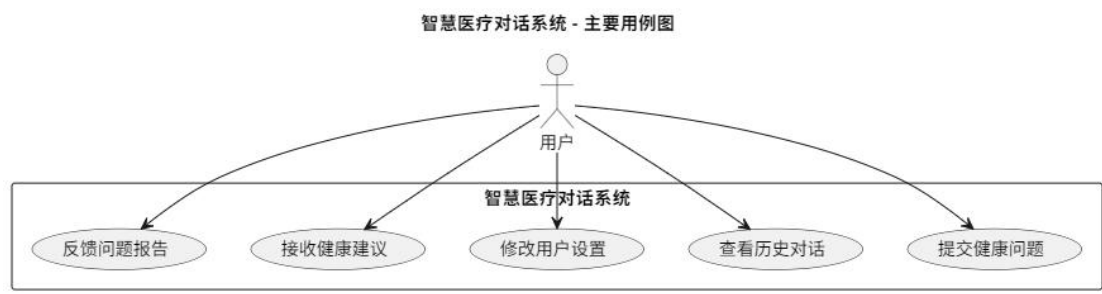
对话系统：智能问答机器人，执行自然语言理解与回复生成。

知识库系统：提供结构化健康医疗知识支持。

8.2 主要用例描述

用例名称	描述	参与者	触发条件	结果描述
提交健康问题	用户输入健康相关问题，系统返回回答	用户	用户在问答区输入并提交问题	系统生成或检索回答，展示给用户
查看历史对话	用户查看之前的问答记录	用户	用户打开历史会话面板	显示历史问答列表，支持筛选和加载上下文
修改用户设置	用户调整个性化参数，如字体大小、配色方案	用户	用户进入设置界面并保存更改	系统保存设置并应用于界面
接收健康建议	系统主动推送相关健康知识和日常管理建议	对话系统	用户对话过程中或登录时	健康辅助模块显示个性化建议
反馈问题报告	用户提交对回答的反馈或错误报告	用户	用户点击反馈入口，填写反馈内容	系统记录反馈，通知管理员

用例图如下：



第九部分：类设计

本系统基于模块化、职责单一、高内聚低耦合的原则进行类设计，划分如下核心类：

UserProfile: 负责维护用户基本信息与偏好设置

DialogueManager: 管理对话上下文与问答流程

LLMInterface: 封装对大语言模型的调用逻辑

KnowledgeBaseQuery: 处理知识库内容的索引与检索

HealthAdviceGenerator: 主动生成健康建议，结合用户状态

SessionHistory: 负责用户历史对话的存取和筛选

FeedbackProcessor: 管理用户反馈和问题报告

类职责说明

UserProfile

属性: userID, nickname, gender, age, preferences

方法: updateProfile(), getPreference(), toggleSetting()

DialogueManager

属性: sessionId, userContext, dialogueHistory

方法: submitQuestion(), getAnswer(), continueSession()

LLMInterface

属性: modelName, temperature, maxTokens

方法: generateResponse(prompt), streamResponse(prompt)

KnowledgeBaseQuery

属性: indexPath, queryStrategy

方法: search(query), retrieveSourceDocs()

HealthAdviceGenerator

属性: userData, healthRules

方法: generateAdvice(), recommendPlan()

SessionHistory

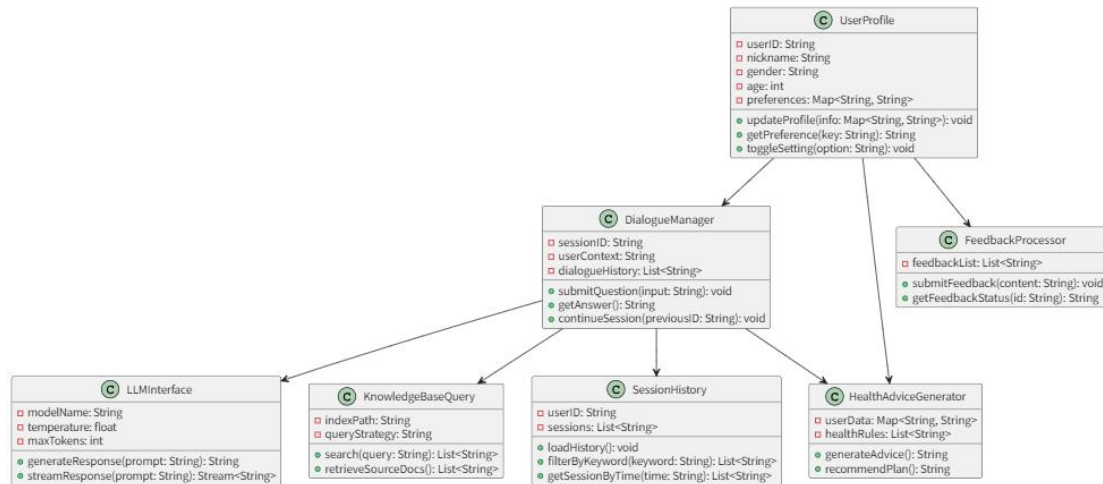
属性: userID, sessions

方法: loadHistory(), filterByKeyword(), getSessionByTime()

FeedbackProcessor

属性: feedbackList

方法: submitFeedback(content), getFeedbackStatus(id)



第十部分：数据设计

10.1 设计原则

结构清晰：实体划分清晰、属性定义规范；

满足扩展性：支持后续添加慢病管理、智能推荐等扩展功能；

兼容多终端会话：支持多端登录、历史记录追踪；

隐私安全：敏感信息如用户信息加密存储。

10.2 核心数据实体及字段

1) 用户信息表（User）

字段名	类型	描述
id	UUID	用户唯一标识
username	String	用户名
password_hash	String	加密后的密码
nickname	String	昵称
age	Integer	年龄
gender	Enum	性别（M/F/Other）
created_at	Timestamp	注册时间

2) 问答记录表 (ChatHistory)

字段名	类型	描述
id	UUID	问答记录唯一标识
user_id	UUID	所属用户 ID
session_id	String	会话 ID (上下文)
question	Text	用户问题
answer	Text	模型回复
source_type	Enum	来源 (知识库/模型)
source_doc_id	UUID	知识库文档 ID
timestamp	Timestamp	提问时间

3) 知识库文档表 (KnowledgeDoc)

字段名	类型	描述
id	UUID	文档 ID
title	String	文档标题
content	Text	文档原文
tags	Array	标签 (糖尿病等)
updated_at	Timestamp	更新时间

4) 健康建议表 (HealthAdvice)

字段名	类型	描述
id	UUID	建议 ID
user_id	UUID	关联用户
advice_type	Enum	类型 (饮食/运动)
content	Text	建议内容
generated_at	Timestamp	生成时间

5) 用户反馈表 (Feedback)

字段名	类型	描述
id	UUID	反馈 ID
user_id	UUID	反馈用户 ID
feedback	Text	反馈内容
resolved	Boolean	是否已处理

字段名	类型	描述
submitted_at	Timestamp	提交时间

10.3 数据关系设计说明

一个用户 (User) 可以有多个问答记录 (ChatHistory) ;

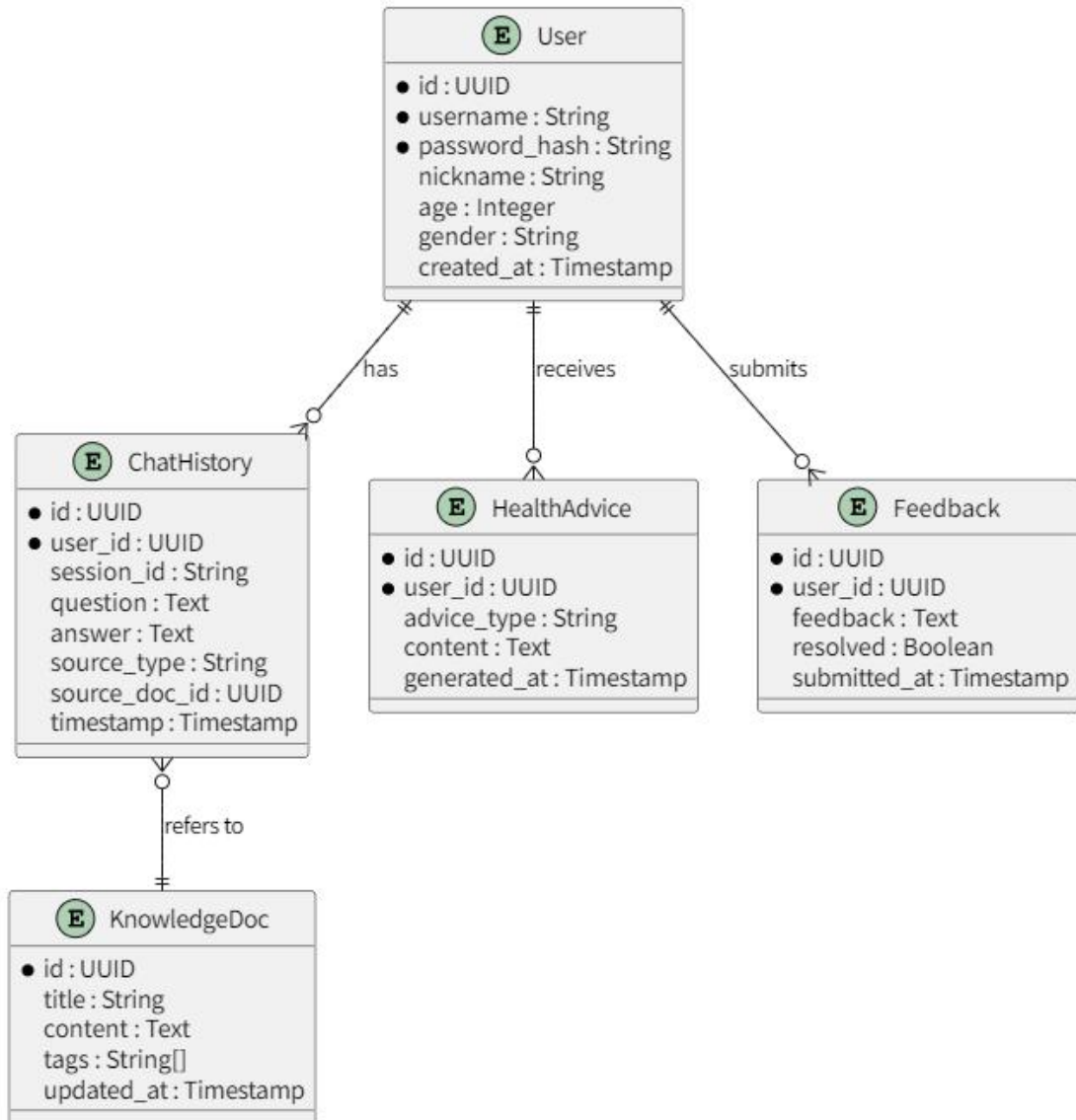
每条问答记录可能关联一个知识库文档 (KnowledgeDoc) ;

用户可以收到多条健康建议 (HealthAdvice) ;

用户可以提交多个反馈 (Feedback) ;

10.4 数据建模 ER 图

智慧医疗对话系统 ER 图



第十一部分：接口设计

11.1 接口设计原则

RESTful 风格：采用统一资源标识符，符合 HTTP 协议设计；

数据标准化：输入输出均为 JSON 格式，字段清晰统一；

安全可控：通过 Token 鉴权机制、接口限流、日志审计保证安全；

模块解耦：前后端、知识库、大模型等模块通过清晰接口通信。

11.2 接口划分与说明

1) 用户管理接口

接口路径	方法	描述
/api/user/login	POST	用户登录
/api/user/info	GET	获取用户信息
/api/user/update	POST	修改用户资料

2) 问答交互接口

接口路径	方法	描述
/api/chat/ask	POST	提交用户提问，返回模型答案
/api/chat/stream	POST	流式对话，适用于长回复
/api/chat/session	GET	获取当前对话上下文
/api/chat/continue	POST	在历史会话上继续提问

3) 知识库接口

接口路径	方法	描述
/api/knowledge/query	POST	查询知识库并返回匹配内容
/api/knowledge/source_docs	GET	获取具体知识出处

4) 健康建议接口

接口路径	方法	描述
/api/health/advice	GET	获取个性化健康建议
/api/health/plan	GET	推荐饮食/运动计划

5) 对话记录接口

接口路径	方法	描述
/api/history/list	GET	获取用户所有对话记录
/api/history/export	GET	导出为文件（PDF/JSON）
/api/history/delete	POST	删除某一条历史记录

6) 反馈接口

接口路径	方法	描述
/api/feedback/submit	POST	提交用户反馈或错误报告

接口路径	方法	描述
/api/feedback/status	GET	获取反馈处理状态

11.3 接口类图



11.4 数据格式示例

用户登录请求

```
POST /api/user/login{
  "username": "zhangsan",
  "password": "123456"}
```

提问接口响应

```
POST /api/chat/ask{
  "answer": "糖尿病患者应控制碳水化合物摄入...",
  "source": "知识库",
  "keywords": ["糖尿病", "饮食"],
  "related": ["饮食建议", "血糖监测"]}
```

第十二部分：总结与展望

本系统结合 Dify 平台与大语言模型,设计了一套完整的智慧医疗对话解决方案,当前聚焦糖尿病问题应答,未来可拓展至多种慢性病管理场景。随着大模型性能提升与知识库建设深入,该系统有望成为辅助医生与患者沟通的重要工具。